



This project may appeal to programmers who dislike the indentation-based structure in CoffeeScript but otherwise like the language for its simplicity, terseness and expressiveness.

Even though CoffeeScript has been influenced by Python and Ruby among other languages, it does not use the colon and the **end** keyword. **ClosingScript** reintroduces both to clearly delimit code blocks.

Even though this adds some visual clutter, the resulting code is still way cleaner and more expressive than JavaScript.

- Some people feel more comfortable with the structural integrity offered by delimited blocks.
- Whitespace is no longer significant.
- Safe copy-pasting without worrying about indentation levels.
- Safe auto-formatting.

Syntax considerations

- Keep clear the line after a trailing colon.
- No comment is allowed after a trailing arrow.
- Only use code blocks with multi-line **if**, **switch** or **try** structures. Don't mix with single line+code.
- The **switch** structure being a strange beast, it must be terminated with the keyword **endsw**.

Usage

ClosingScript is a text preprocessor that outputs to **stdout**. The CoffeeScript compiler must be installed to get JavaScript.

closing <option> <script>.coffee

Options:

-c	convert code	# replace braces with equivalent indentation
-f	format code	# reindent lines consistently, code is untouched

Typical usage

```
# Convert and compile to JavaScript in one step
closing -c script.coffee | coffee -s -c > script.js

# Get a converted copy of a source
closing -c script.coffee > script.converted.coffee

# Get a formatted copy of a source
closing -f script.coffee > script.formatted.coffee
```

General syntax

```
functionName = (parms) ->  
  code  
end
```

```
functionName = (parms) -> code
```

```
do ->  
  code  
end
```

```
do -> code
```

```
[variable =] if <condition>:  
  code  
else if <condition>:  
  code  
else:  
  code  
end
```

```
[variable =] if <condition> then code [else code]
```

```
code if <condition>
```

```
[variable =] while <condition> [guard clauses]:  
  code  
end
```

```
[variable =] while <condition> code [guard clauses] then code
```

```
[variable =] for <iterable> [guard clauses]:  
  code  
end
```

```
[variable =] for <iterable> code [guard clauses] then code
```

```
[variable =] switch [variable]:  
  when <condition>:  
    code  
  [when <condition>:  
    code]  
  [else:  
    code]
```

```
endsw
```

use special keyword **endsw** to close a **switch** stucture

```
try:  
  code  
[catch [var]:  
  code]  
[finally:  
  code]  
end
```

```
class name:  
  code  
end
```